



Transformer 구현

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Self-Attention과 Transformer

◆ Transformer Block 내부 구조

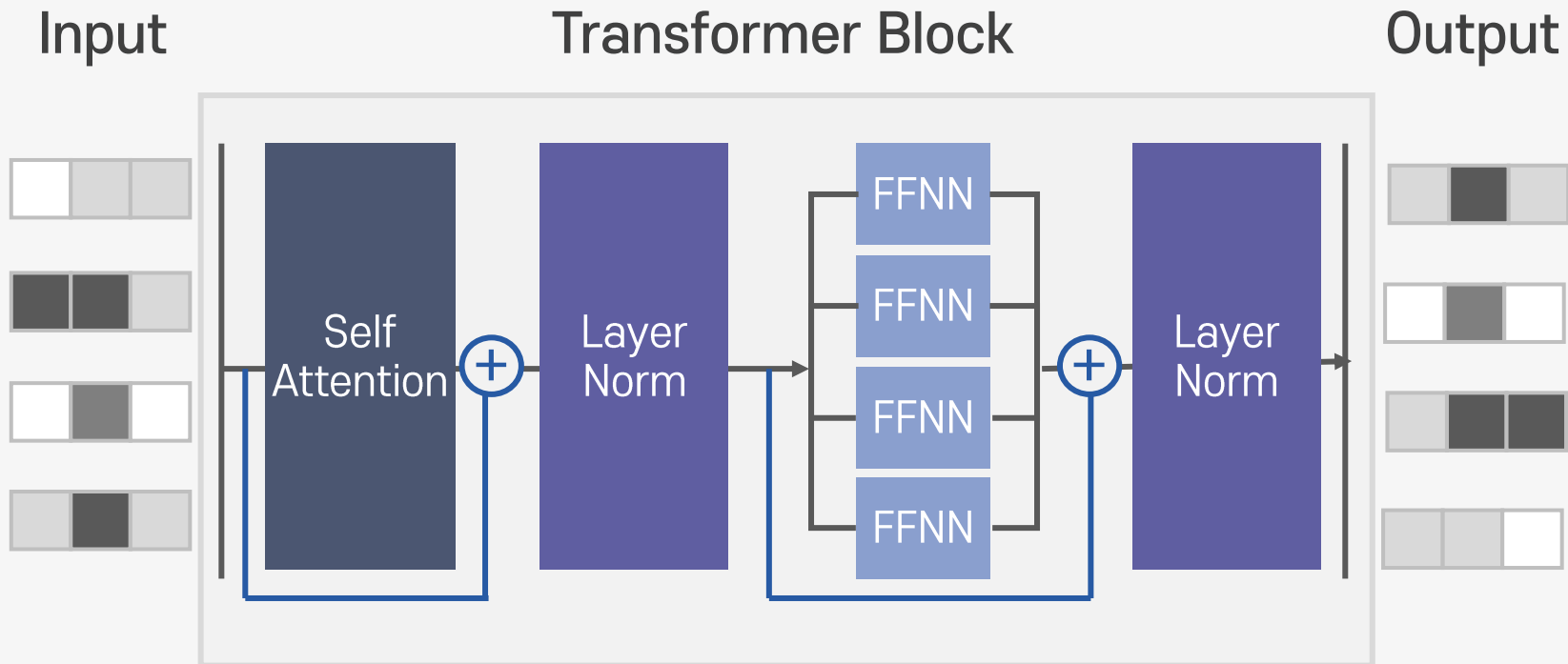
Self Attention, Normalization Layer#1,
Feed Forward Neural Network, Normalization Layer#2 및
Residual Network(Skip Connection) 등으로 구성

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Self-Attention과 Transformer

Transformer Block 내부 구조



자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Self-Attention과 Transformer

◆ Query, Key, Value 계산

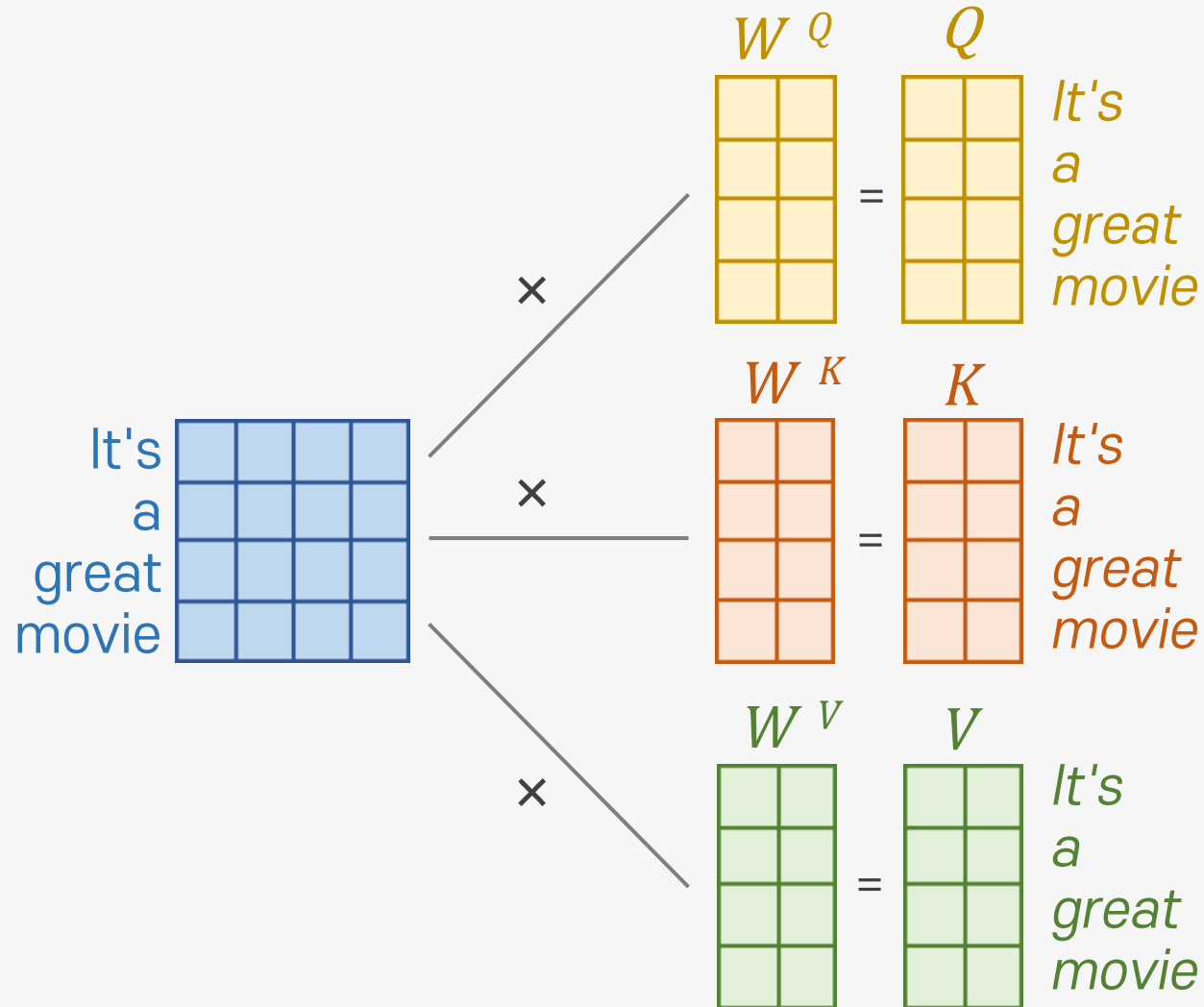
Self-Attention의 첫 단계는 입력 문장에 대한 Query, Key, Value 계산

Input 문장의 512크기의 벡터와 학습할 Weight(W_Q , W_K , W_V)를 곱하여 64크기의 Query, Key, Value 벡터 생성

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Self-Attention 구현



자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Self-Attention 구현

◆ Query, Key, Value 계산

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
    b, l, d = x.size()
```

```
    h = self.h
```

```
    queries = self.WQ(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, l, d)
```

```
    keys = self.WK(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, l, d)
```

```
    values = self.WV(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, l, d)
```

```
    w_prime = torch.bmm(queries, keys.transpose(1, 2)) / np.sqrt(d)
```

```
    w = F.softmax(w_prime, dim=-1)
```

```
    out = torch.bmm(w, values).view(b, h, l, d)
```

```
    out = out.transpose(1, 2).contiguous().view(b, l, h * d)
```

```
    return self.unifyheads(out)
```

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Self-Attention 구현

◆ Query, Key, Value 계산

Query에 Key의 Transpose한 행렬을 내적(Dot Product)

Query와 Key가 특정 문장에서 중요한 역할을 하고 있는 경우
Transformer는 이들 사이의 내적(Dot Product) 값을 크게 하는
방향으로 학습

내적 값이 커지면 해당 Query와 Key가 벡터 공간 상
가까이에 있을 확률 높음

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Self-Attention 구현

◆ Self-Attention의 계산

The diagram illustrates the calculation of Self-Attention. It shows a yellow matrix Q (labeled "It's a great movie") multiplied by an orange matrix K^T (labeled "It's a great movie"). The result is a green matrix (labeled "It's a great movie").

Q

K^T

$\times$

$=$

+ Self-Attention 구현

Self-Attention의 계산

```
def forward(self, x: torch.Tensor) -> torch.Tensor:

    b, l, d = x.size()
    h = self.h

    queries = self.WQ(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, l, d)
    keys = self.WK(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, l, d)
    values = self.WV(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, l, d)

    w_prime = torch.bmm(queries, keys.transpose(1, 2)) / np.sqrt(d)
    w = F.softmax(w_prime, dim=-1)

    out = torch.bmm(w, values).view(b, h, l, d)

    out = out.transpose(1, 2).contiguous().view(b, l, h * d)

    return self.unifyheads(out)
```

+ Self-Attention 구현

◆ Self-Attention의 계산

Key의 벡터 크기인 64의 root 값인 8로 나눈 후
softmax 함수 적용

Value와 내적(Dot Product)을 곱하여 Attention Value인 Z 계산

- $\sqrt{d_k}$ 로 나눈 이유는 Query와 Key의 내적 행렬의 분산을 축소하고
gradient vanishing 발생 방지

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Self-Attention 구현

Self-Attention의 계산

Diagram illustrating the Self-Attention calculation process:

Input matrices Q (4x2, yellow) and K^T (2x4, orange) are multiplied together. The result is then passed through a softmax function, scaled by $\sqrt{d_k}$. This result is then multiplied by the input matrix V (4x2, green) to produce the final Attention Value Matrix α (4x2, blue).

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Self-Attention 구현

◆ Self-Attention의 계산

```
def forward(self, x: torch.Tensor) -> torch.Tensor:

    b, l, d = x.size()
    h = self.h

    queries = self.WQ(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, l, d)
    keys = self.WK(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, l, d)
    values = self.WV(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, l, d)

    w_prime = torch.bmm(queries, keys.transpose(1, 2)) / np.sqrt(d)
    w = F.softmax(w_prime, dim=-1)

    out = torch.bmm(w, values).view(b, h, l, d)

    out = out.transpose(1, 2).contiguous().view(b, l, h * d)

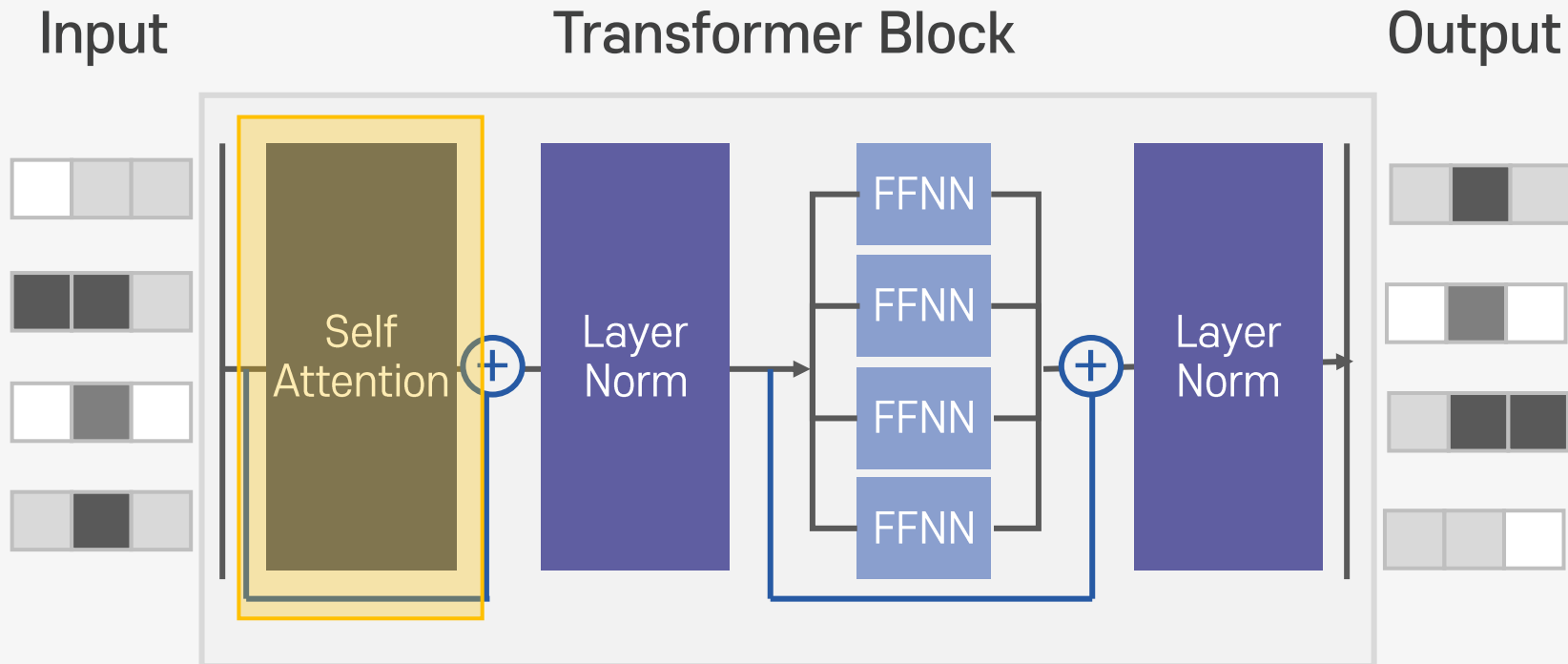
    return self.unifyheads(out)
```

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Transformer 구현

Self-Attention 추가



자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Transformer 구현

◆ Self-Attention 추가

```
class TransformerBlock(nn.Module):  
    def __init__(self, d: int, heads: int=8, n_mlp: int=4):  
        super().__init__()  
  
        self.attention = SelfAttention(d, heads=heads)  
        self.norm1 = nn.LayerNorm(d)  
        self.norm2 = nn.LayerNorm(d)  
  
        self.ff = nn.Sequential(  
            nn.Linear(d, n_mlp*d),  
            nn.ReLU(),  
            nn.Linear(n_mlp*d, d)  
        )
```

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Transformer 구현

◆ Self-Attention 추가

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    x_prime = self.attention(x)
    x = self.norm1(x_prime + x)

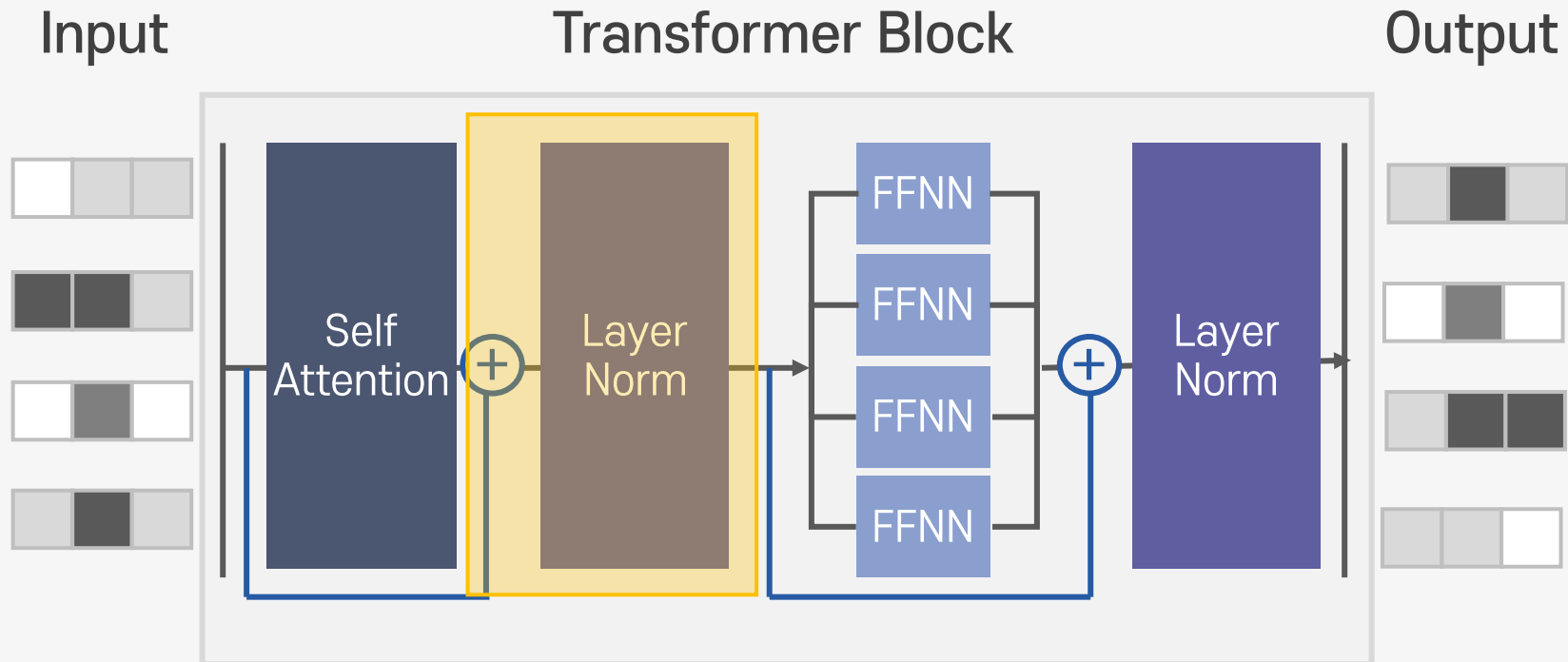
    x_prime = self.ff(x)
    return self.norm2(x_prime + x)
```

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Transformer 구현

Self-Attention 추가



자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Transformer 구현

◆ Normalization Layer 추가

```
class TransformerBlock(nn.Module):
    def __init__(self, d: int, heads: int=8, n_mlp: int=4):
        super().__init__()

        self.attention = SelfAttention(d, heads=heads)
        self.norm1 = nn.LayerNorm(d)
        self.norm2 = nn.LayerNorm(d)

        self.ff = nn.Sequential(
            nn.Linear(d, n_mlp*d),
            nn.ReLU(),
            nn.Linear(n_mlp*d, d)
        )
```

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Transformer 구현

◆ Normalization Layer 추가

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    x_prime = self.attention(x)
    x = self.norm1(x_prime + x)

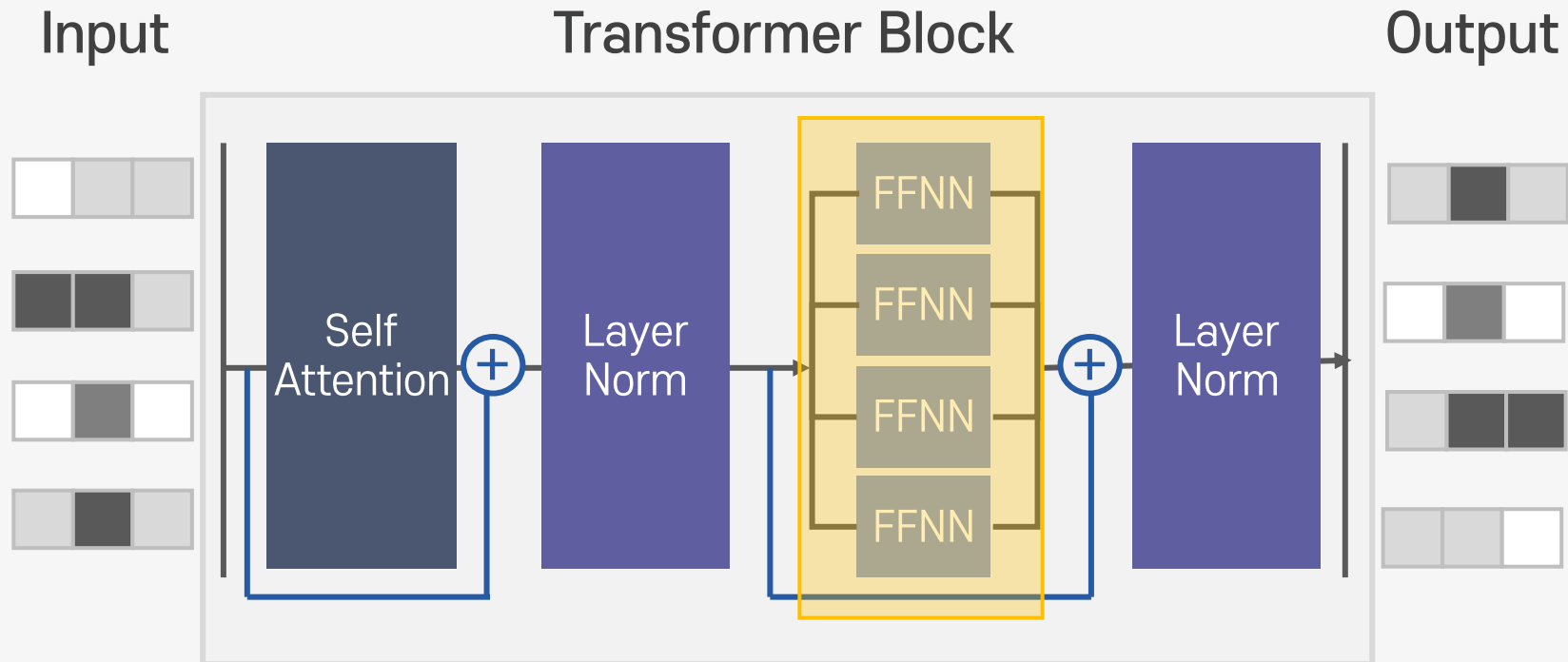
    x_prime = self.ff(x)
    return self.norm2(x_prime + x)
```

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Transformer 구현

◆ Feed Forward Neural Network 추가



+ Transformer 구현

◆ Feed Forward Neural Network 추가

```
class TransformerBlock(nn.Module):  
    def __init__(self, d: int, heads: int=8, n_mlp: int=4):  
        super().__init__()  
  
        self.attention = SelfAttention(d, heads=heads)  
        self.norm1 = nn.LayerNorm(d)  
        self.norm2 = nn.LayerNorm(d)  
  
        self.ff = nn.Sequential(  
            nn.Linear(d, n_mlp*d),  
            nn.ReLU(),  
            nn.Linear(n_mlp*d, d)  
        )
```

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Transformer 구현

◆ Feed Forward Neural Network 추가

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    x_prime = self.attention(x)
    x = self.norm1(x_prime + x)

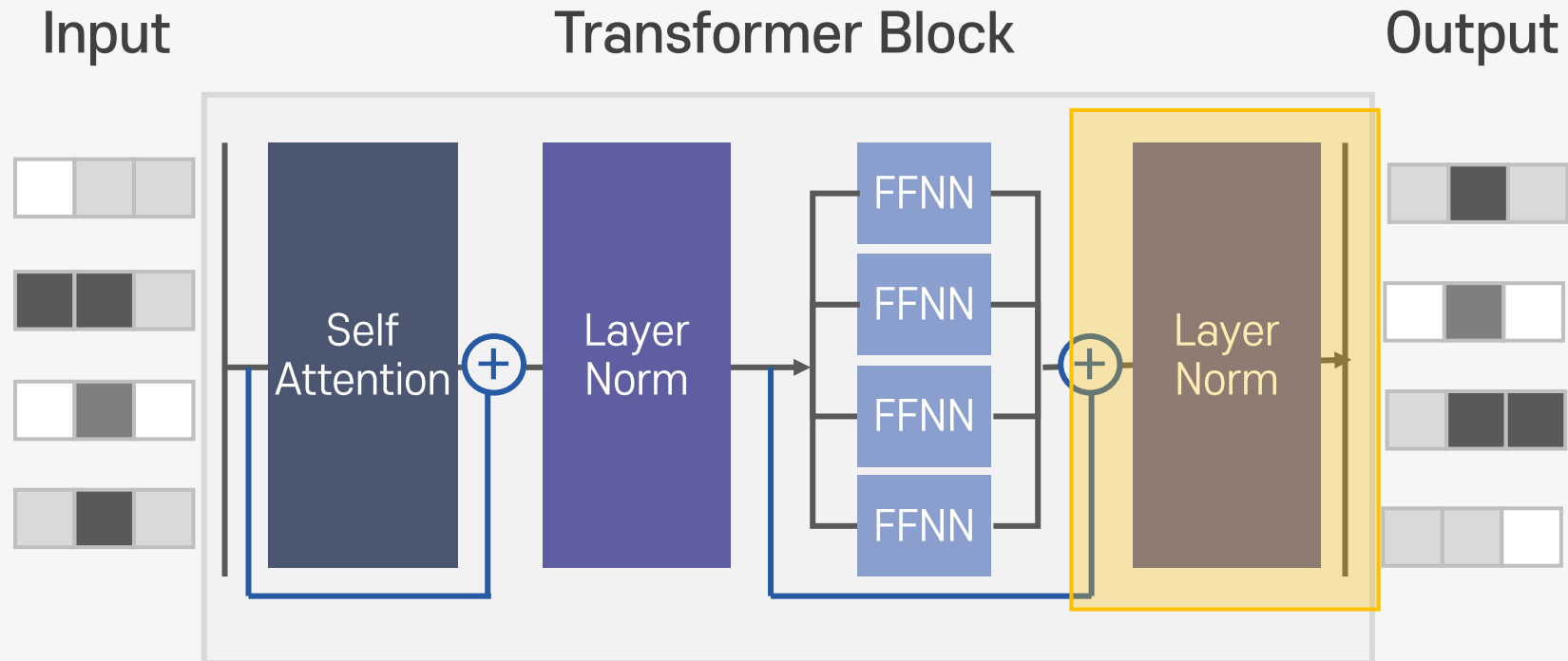
    x_prime = self.ff(x)
    return self.norm2(x_prime + x)
```

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Transformer 구현

◆ Normalization 추가



+ Transformer 구현

◆◆ Normalization Layer추가

```
class TransformerBlock(nn.Module):
    def __init__(self, d: int, heads: int=8, n_mlp: int=4):
        super().__init__()

        self.attention = SelfAttention(d, heads=heads)
        self.norm1 = nn.LayerNorm(d)
        self.norm2 = nn.LayerNorm(d)

        self.ff = nn.Sequential(
            nn.Linear(d, n_mlp*d),
            nn.ReLU(),
            nn.Linear(n_mlp*d, d)
        )
```

자연어 처리를 위한 Transformer 적용

Attention과 Transformer 구현

+ Transformer 구현

◆ Self-Attention 추가

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    x_prime = self.attention(x)
    x = self.norm1(x_prime + x)

    x_prime = self.ff(x)
    return self.norm2(x_prime + x)
```


자연어 처리를 위한 Transformer 적용

Transformer 감성분석 적용

+ Transformer 전체 구조

◆ Transformer 적용을 위한 방법

Word embedding과 Positional Embedding 필요

Transformer를 멀티 Layer로 적용

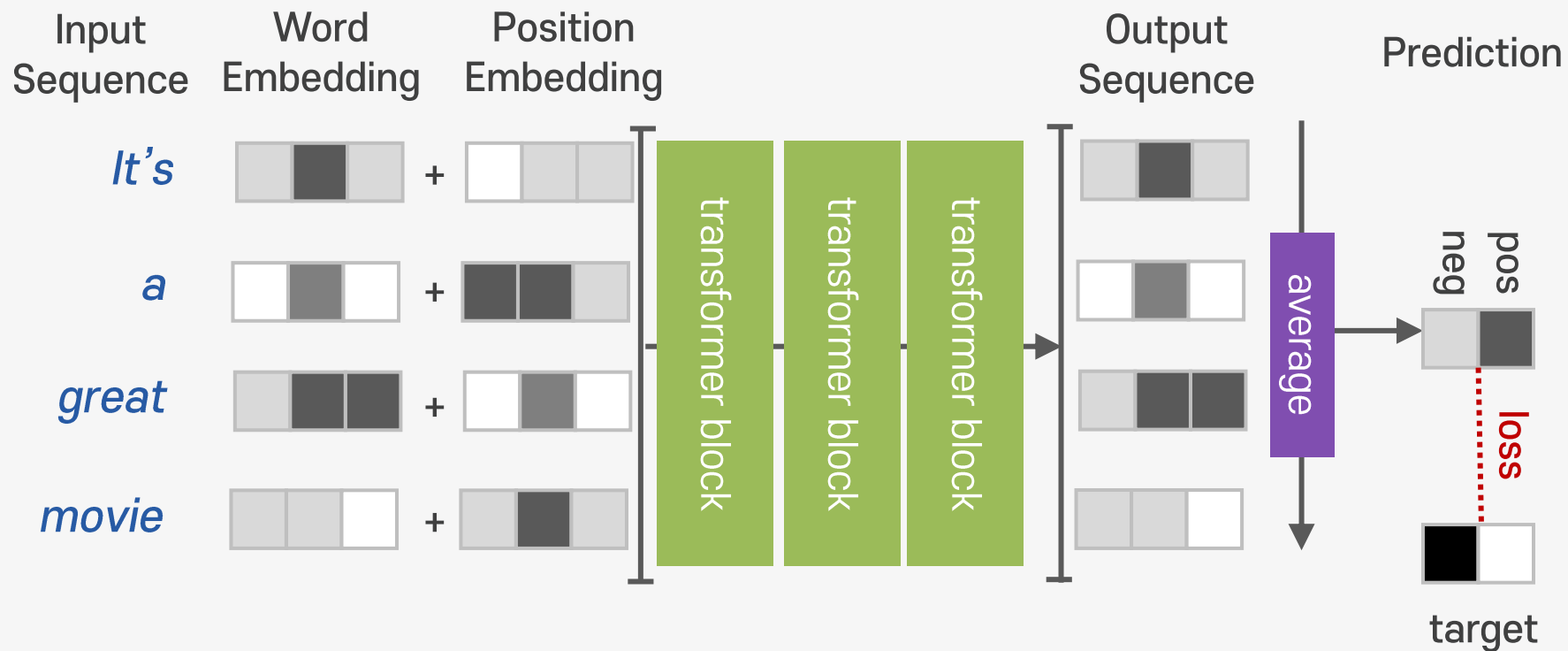
긍정/부정 학습 및 예측을 위한 분류기 추가

자연어 처리를 위한 Transformer 적용

Transformer 감성분석 적용

+ Transformer 전체 구조

◆ Transformer 적용을 위한 방법

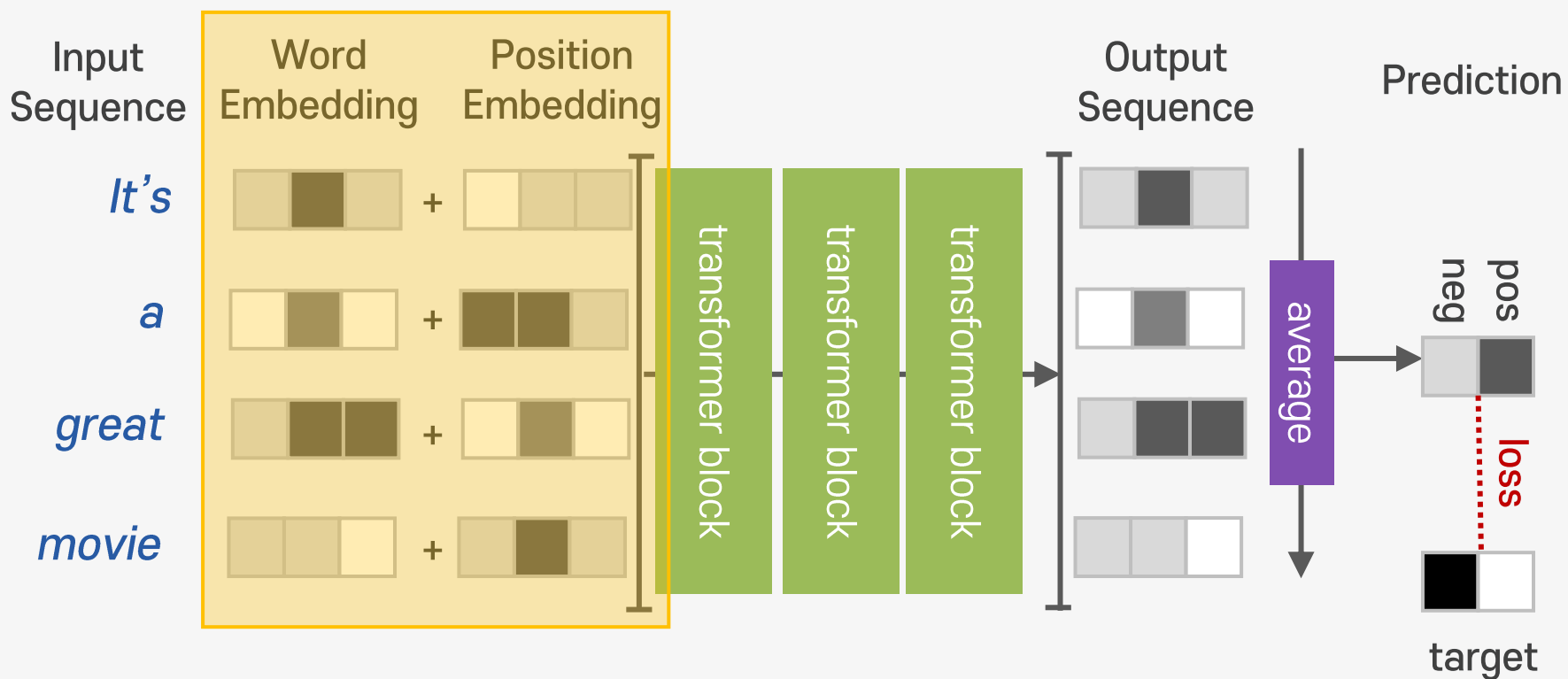


자연어 처리를 위한 Transformer 적용

Transformer 감성분석 적용

+ Transformer 적용

◆ Word Embedding과 Positional Embedding Layer 추가



자연어 처리를 위한 Transformer 적용

Transformer 감성분석 적용

+ Transformer 적용

◆▶ __init__

```
self.token_emb = nn.Embedding(num_tokens, d)
self.pos_emb = nn.Embedding(max_seq_len, d)
```

◆▶ __forward__

```
tokens = self.token_emb(x)
positions = self.pos_emb(torch.arange(1).to(self.device)).expand(b, 1, d)
embeddings = tokens + positions
```

자연어 처리를 위한 Transformer 적용

Transformer 감성분석 적용

+ Transformer 적용

◆◆ Positional Embedding을 추가하는 이유?

◆ RNN + Attention

- RNN제거 후 Attention 만으로 Transformer 구현

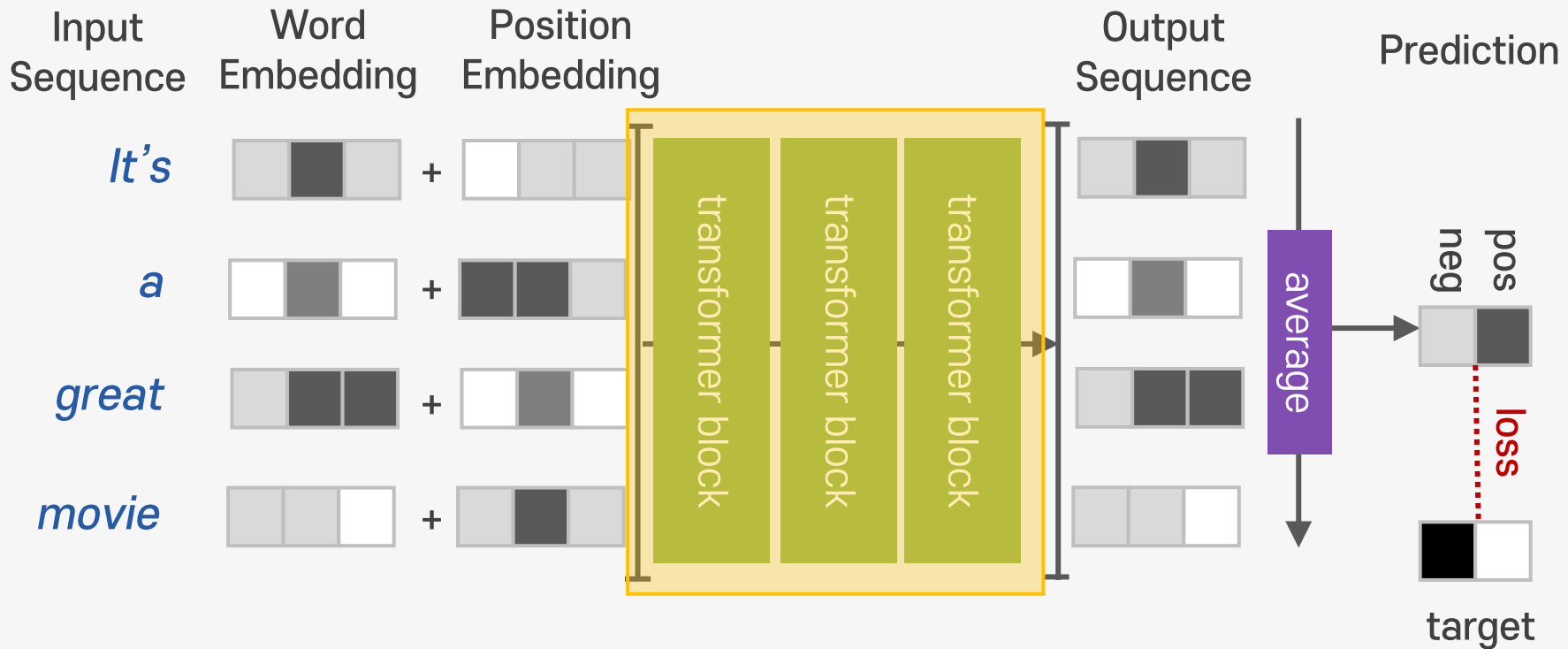
◆ RNN이 제거되어 토큰의 별도 순서 지정 필요

자연어 처리를 위한 Transformer 적용

Transformer 감성분석 적용

+ Transformer 적용

◆ Transformer Block 추가



자연어 처리를 위한 Transformer 적용

Transformer 감성분석 적용

+ Transformer 적용

◆▶ `__init__`

```
self.transformer_blocks = nn.Sequential(  
    *[TransformerBlock(d=d, heads=heads) for _ in range(depth)]  
)
```

◆▶ `__forward__`

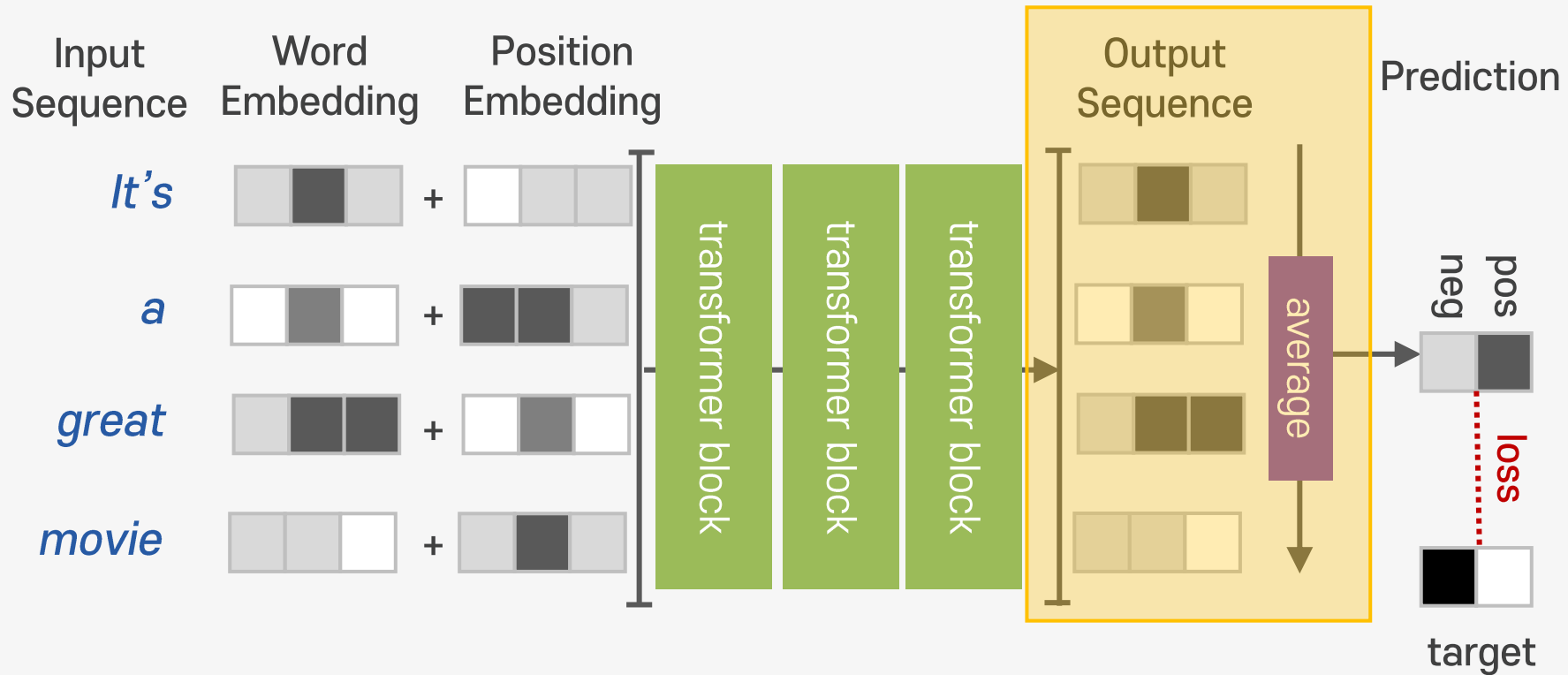
```
out = self.transformer_blocks(embeddings)
```

자연어 처리를 위한 Transformer 적용

Transformer 감성분석 적용

+ Transformer 적용

◆◆ Classification Layer 추가



자연어 처리를 위한 Transformer 적용

Transformer 감성분석 적용

+ Transformer 적용

◆▶ `__init__`

```
self.classification = nn.Linear(d, num_classes)
```

◆▶ `__forward__`

```
out = out.mean(dim=1)  
out = self.classification(out)
```